

Final Project Report

Team: William Dappen, John Jezl, Angelica Rohrs

Abstract

In a string-matching game, two players compete by each selecting a target string. Random symbols are drawn uniformly from an alphabet, and the first player whose target string appears as a contiguous substring wins. Computing the exact probability that a given player wins requires modeling the game as a Markov process. Our solution constructs a Deterministic Finite Automaton (DFA) to track game states, builds a system of recurrence equations representing the win probabilities from each state, and solves the resulting linear system using Gauss-Jordan elimination with exact rational arithmetic. The output is the probability P/Q in lowest terms that Player A wins the game.

Problem Statement

Given two strings x and y of the same length N over a finite alphabet Σ of size M . (N and M are user inputs so your solution should work for arbitrary N and M .) Consider a game involving two players A and B. A random string over the alphabet Σ is generated until x or y appears as a substring. (You can assume that x and y do not have a common prefix other than the null string. Or equivalently, assume that the first letters of x and y are different.) If the game terminates with the appearance of x (y), player 1 (2) wins. You are to write a program that takes as input the strings x and y , and outputs the probability that A wins. For example: suppose $\Sigma = \{0, 1, 2\}$, $x = 00$ and $y = 12$. In this case, the probability of winning for A is $3/7$. We have presented a technique for calculating probability of an event using DFA. Your solution involves two steps: (i) write a program that takes as input the strings x and y and outputs a DFA D that has two special terminal states $T1$ and $T2$ with no transitions such that any string that has a unique occurrence of x as suffix and no occurrence of y ends in state $T1$ and any string that has a unique occurrence of y as suffix and no occurrence of x ends in state $T2$. (b) write a program that takes as input the DFA D and computes the probability of win for 1. This involves generating the recurrence equations and using a standard library to solve them.

Problem Background

We represented this problem as a Markov chain where each state represents the current "progress" toward a player's target string. The probability of winning from any state depends only on the current state, not any previous states reached.

We split the solution into three parts:

1. **DFA Construction** - Build an automaton that tracks the longest suffix of the current string that could result in a player winning
2. **Recurrence Equation Builder** - For each non-absorbing state, represent the probability of winning as a linear combination of win probabilities from successive states.
3. **Linear System Solver** - Solve the equations using rational math libraries that handle large scale values to compute a precise probability.

The game always terminates (one player will win), and the probability can be computed exactly as a rational number P/Q.

Algorithms Used

DFA

To build the DFA for this project, I use a simplified version of the Knuth-Morris-Platt algorithm's core idea. Since we only need to track two possible target strings, we can create states for every prefix of each target string and then define transitions using the KMP-style longest-suffix rule. For any current state prefix and incoming character, we attempt to extend the current prefix using the input character. If this direct extension fails, we repeatedly remove characters from the front until the remaining suffix matches a known prefix of either target string; that suffix becomes the next state. This new state captures exactly how far we've progressed toward either pattern at any moment.

This approach works cleanly for our game because the patterns do not share a prefix and the fallback structure remains straightforward. As soon as the DFA reaches a state equal to one of the full target strings, we transition into an absorbing state indicating that the game is over and that pattern as the winner. Even though the method is much simpler than AC, it is logically consistent with the rules of the game and guarantees a complete, deterministic automaton that correctly models which string appears first.

Recurrence Equation Builder

For each non-absorbing state, we built the recurrence equation

$$P(q) = (1/M) \times \sum P(\delta(q, a))$$

for all symbols in the alphabet (Σ). This represents the probability of A winning from a given state. Since each symbol is drawn at random with probability $1/M$, the probability of winning from q is the average of the probabilities of winning from each possible next state.

There were also two boundary conditions, where $P(x)$ is 1 (player A wins) and $P(y)$ is 0 (player B wins).

Solver

The Gauss-Jordan elimination algorithm was used to solve the system of linear equations in the function `solveLinearSystem`. When given A , an $n \times n$ matrix of coefficients for each equation, and b , an $1 \times n$ vector of constants for each equation, it does $A * x = b$ to solve for x , which is a vector containing the solutions to each equation. To solve, first it builds an augmented matrix $[A \mid b]$ using two nested for loops. Then it starts the Gauss-Jordan elimination for each column by finding the non-zero pivot in/under the row number matching the column number and swaps the pivot row into place. The pivot row gets normalized, then the column is eliminated in all other rows. Once the Gauss-Jordan elimination is finished, the augmented matrix $[A \mid b]$ has been transformed into $[I \mid x]$, where I is the identity matrix, so the last column in the matrix, which contains the solutions, is copied over to a vector and returned.

In the `Rational` struct used by the equation builder and solver, a Euclidean GCD (Greatest Common Denominator) algorithm is used as part of the process of simplifying fractions. It works by taking the absolute value of the numerator and denominator, finding the remainder, r , when the numerator, a , is divided by the denominator, b , with $r = a \% b$, then making $a = b$ and $b = r$ and repeating until b is equal to 0. At that point, a is the GCD, so it is returned. In the function that simplifies fractions, the numerator and denominator are simplified by dividing by the GCD if it is not zero.

Data Structures Used

DFA

For the constructed DFA, three data structures were used to quickly access each relevant piece of data in the DFA. Firstly, a vector of `State` objects was used to represent each state, as well as its corresponding ID, classified by its vector offset. The `State` object contains a label, in this case the specific substring state the head is in, as well as its classification as an absorbing state, which is used to determine whether or not the game has ended.

This vector was complemented by an unordered map, called '`State_Lookup`,' which maps a substring to its `StateID`, a number corresponding to the vector offset of the given substring. These two structures were created in tandem with each other to allow for proper indexing of both structures.

Finally, transitions between states were mapped with a vector containing unordered mappings of characters to integers. This system works by representing each state as an item on the vector, after which each possible inputted character represents the transition character, which points to an integer representing the vector offset of the target state. This structure is created and filled after the first two, acting as the bulk of the logic in the DFA.

Recurrence Equation Builder

Since the form of the equation is fixed, only the coefficients need to be stored. And since the number of items in the matrix is $N \times N$, but there will only be non-zero coefficients in $N \times M$ positions in the matrix, a sparse representation was used. These are stored in COO format as tuples of (row, column, value). This saves memory as well as compute overhead.

We also keep a map from state IDs to equation row indices to allow lookup of the solution for any state.

Solver

For the recurrence equation solver, vectors and 2D vectors were used as matrices. In order to obtain precise fractional answers, a Rational struct was created, which stores each number as a numerator and denominator. The number zero is stored as numerator = 0 and denominator = 1. An external library called Boost was used for arbitrarily large integers, which is the type that the numerators and denominators are stored as. The Rational type has various functions that allow arithmetic to be done with Rational types.

Time Complexities of Algorithms

Algorithm	Complexity	Notes
DFA Construction	$O(N^2 \times M)$	Mostly within <code>Handle_fail_cases()</code> , reduced from $O(N^3 \times M)$
Recurrence Equation Builder	$O(N \times M \log M)$	Iterates states over Σ with map operations
Solver	$O(K \times N^3)$	Gauss-Jordan algorithm
Overall	$O(K \times N^3)$	Solver becomes dominant at large N values

N = Number of DFA states ($\approx |x| + |y| + 1$)

M = Alphabet size ($|\Sigma|$)

K = Number of digits in the rational numbers

DFA

During the development of this assignment, I discovered that my original `handle_fail_cases()` function had a very poor time complexity. Because it repeatedly rebuilt strings using `substr` and `erase`, and performed many unordered-map lookups on those temporary strings, the overall construction cost reached $O(N^3 \times M)$, where N is the total length of the combined input patterns and M is the alphabet size. To address this, I replaced all of the expensive string reconstruction steps with a simple sliding-index technique that produces the same logical effect but avoids repeated character shifting and reallocation. After this optimization, each fallback scan runs in

O(N), reducing the total DFA-building time to **O(N² × M)**. This performance improvement is essential to keep the DFA construction efficient for the input sizes required by the assignment.

Recurrence Equation Builder

Initialization of the builder is a simple O(n) complexity (loops through the states). As is `apply_boundary_conditions`. `generate_equation_for_state` is where the bulk of the operations occur. Row index map lookups consume O(log N) and looping through M symbols with map lookups of O(log M), is O(M log M), a larger value. Since this is called N times from the calling function, this results in a time complexity of **O(N * M log M)**

Solver

In the Rational type, the Euclidean GCD algorithm is **O(n²)**. The number of iterations in the Euclidean algorithm is O(n) since numbers can be arbitrarily large and modulo on BigInt (arbitrarily large integers) is O(n²). Putting them together, O(n) * O(n²) would give O(n³), but the size of the number shrinks each time during the modulo operation, so the real time complexity is less than O(n³) and is more like O(n²).

The Gauss-Jordan algorithm is **O(n³)**. There are three phases per pivot. First, finding the pivot is O(n). Second, normalizing the pivot row (dividing around n entries) is O(n), and third, eliminating the pivot column from all other rows is O(n²) because each row does around n operations, and there are n rows. Doing it for each of the n pivots, the total is n x (O(n)+O(n)+O(n²)) = O(n³).

However, because of the use of the Rational library, each operation has a cost of K, where K is the number of digits in the value, making the final time complexity **O(KN³)**

Program Implementation

The program asks the user to input a string for player A, a string for player B, and an alphabet size up to 32. The minimum alphabet size for the input strings is calculated and if it is impossible to create an alphabet 32 or less with them, the program outputs an error and ends. Otherwise, the alphabet size is checked to make sure it is at least as large as the minimum alphabet size and less than or equal to 32. If not, the user is prompted to reenter the alphabet size until it satisfies those conditions. Once a good alphabet size is chosen, a function called `setUpAlphabet` sets up the alphabet by placing each unique character in strings A and B into an unordered set that represents the alphabet, then fills the rest of the set with arbitrary characters until the set contains as many characters as the alphabet size. This works because only the characters in the strings are important to the probability calculation. It doesn't matter what the other characters are, as long as they are unique and there are enough of them so that the desired alphabet size is reached.

Next, a GameDFA is created using the two player strings and the alphabet, then a RecurrenceEquationBuilder is created using that GameDFA. The last step is to call a function called `getProbabilityOfWin`, which uses the start state of the GameDFA and the

RecurrenceEquationBuilder to determine the probability of player A winning, which is output to the screen in the form of an exact fraction. This works by creating a matrix of the coefficients and a vector of the constants for the recurrence equations generated by RecurrenceEquationBuilder, which get solved by the solveLinearSystem function.

Summary of Results Obtained

Sample 1

Enter a string for player A: 01

Enter a string for player B: 1

Enter the size of the alphabet (32 or less): 3

The probability of A winning with 01 is $\frac{1}{3}$

Sample 2

Enter a string for player A: qwertyuiopasdfghjklzxcvbnm

Enter a string for player B: 123456

Enter the size of the alphabet (32 or less): 32

The probability of A winning with qwertyuiopasdfghjklzxcvbnm is $\frac{1}{1267650600228229401496703205377}$

Sample 3

Enter a string for player A: a

Enter a string for player B: b

Enter the size of the alphabet (32 or less): 2

The probability of A winning with a is $\frac{1}{2}$

Conclusion

This project demonstrates the use of a DFA to solve a probabilistic problem. By modeling the game as a DFA, we made a complicated probability question into a system of linear equations that can be solved quickly. The three-part approach (DFA, equation building, and solver) allowed the team to work in parallel. The use of rational arithmetic with arbitrary length integers ensured our results are accurate, avoiding the rounding errors inherent in floating-point computation.

Challenges we overcame include:

- Correctly mapping DFA state IDs to equation indices
- Handling edge cases like equivalent prefixing
- Maintaining numerical precision for large alphabets and long strings

The final version computes P/Q in reduced terms for any valid input within constraints, verified against known results such as the simplest case of a probability of 1/2 (Sample 3).

References

Sipser, M. (2012). *Introduction to the Theory of Computation*. Cengage Learning.

Ravikumar, B. (2025, November 27). *discussion of Problem 6 of final project* [Video]. Pizza.
<https://SonomaState.zoom.us/rec/share/kj8MFexle2NXcgNnUSASDQpFMsugdTa4zMc5DowCBfzD5lh3SLeIHFYIQLD1rbiTiZBdIoBeirYMuN-Q>

Ravikumar, B. (2025, November 7). *final project part 2* [Video]. Pizza.
https://SonomaState.zoom.us/rec/share/qFNZ1GRGzH-1mmXu7tgCJIZjZMtczBPWEPwF0TKx-LopVeUW7Kr5YeOXp866aTNO.xJ3_cgpf_5r54Oel

Boost C++ Libraries. (n.d.). Boost.Multiprecision.
<https://www.boost.org/doc/libs/release/libs/multiprecision/>